

✓ Test

🔧 Contexte : Application bancaire orientée objet

Vous allez modéliser un système bancaire modulaire et testable. Il doit suivre les principes de l'encapsulation, de l'héritage, de l'abstraction, et de la sécurisation concurrentielle.

🧱 Partie 1 – Programmation Orientée Objet

◆ 1.1 – Interface ICompte

Créez une interface `ICompte` qui possède les signature suivante :

```
def depot(self, montant: float)
def retrait(self, montant: float)
def afficher(self) -> str
```

Toutes les classes de compte devront implémenter cette interface.

◆ 1.2 – Classe CompteBancaire

Implémente la classe `CompteBancaire` à partir de `ICompte` :

- Attributs : `_titulaire: str`, `_solde: float`
 - Méthodes :
 - `depot(montant: float)`
 - `retrait(montant: float)` (avec exception si fonds insuffisants)
 - `afficher() -> str` (ex: "Alice - Solde : 200.0€")
-

◆ 1.3 – Classe CompteEpargne

- Hérite de `CompteBancaire`
- Ajoute un taux d'intérêt (`_taux_interet`)

- Méthode supplémentaire : `ajouter_interets()`
-

◆ 1.4 – Classe `Client`

- Attributs : `nom: str`, `comptes: list[ICompte]`
 - Méthodes :
 - `ajouter_compte(compte: ICompte)`
 - `resume_comptes()` : affiche tous les comptes du client
-

Partie 2 – Tests unitaires

- Vérifiez les comportements suivants :
 - Ajout, dépôt et retrait sur `CompteBancaire`
 - Application des intérêts sur `CompteEpargne`
 - Affichage correct avec `afficher()`
 - Cas d'exception (fonds insuffisants)
 - Bonus : utilisez `setUp()` pour créer les objets
-

Partie 3 – Multithreading

◆ 3.1 – Version thread-safe de `CompteBancaire`

- Ajoutez un `threading.Lock`
- Protégez les méthodes `depot()` et `retrait()` avec un verrou

◆ 3.2 – Simulation de dépôts concurrents

- 5 threads font chacun 1000 dépôts de 10€
- Affichez le solde attendu et réel

◆ 3.3 – Simulation de retraits concurrents

- Sans verrou : montrez les erreurs
 - Avec verrou : corrigez-les
-

Bonus – Méthode de transfert

- Implémentez `transfert_vers(autre_compte: ICompte, montant: float)` dans `CompteBancaire`
- Gère le retrait + dépôt sécurisé
- Testez cette méthode, y compris en multithread